



RSCAT: Towards zero touch congestion control based on actor–critic reinforcement learning and software-defined networking

Gustavo Diel^{*}, Charles Christian Miers, Maurício Aronne Pillon, Guilherme Piêgas Koslovski^{*}

Graduate Program in Applied Computing (PPGCAP), Santa Catarina State University (UDESC), Brazil

ARTICLE INFO

MSC:
00-01
99-00

Keywords:

Congestion control
Network
Actor–critic
Deep reinforcement learning
Classification
Zero touch

ABSTRACT

Network congestion is a phenomenon present in contemporaneous data centers (DCs) independently of scale and underlying technologies. The small-scale presence of congestion causes information delay for the DC hosted services, while in large-scale data losses or even service downtimes are not uncommon. Several congestion control algorithms out in the market have no network support, and as such, they must act and rely only on the data provided by the servers they are running on. Some algorithms have used network's feedback to improve the performance, however these solutions depend on tuning parameters for marking queued packets, an arduous task on DCs shared by multiple applications with different workloads. In turn, Software-Defined Networking (SDN) created an opportunity to avoid congestion once the centralized controller can gather ongoing and historical information from all network switches and flows. The data gathered is enormous, and fast-computing algorithms are crucial for decision-making. In this sense, this work proposes Reinforcement Learning and SDN-aided Congestion Avoidance Tool (RSCAT), which uses data classification to determine if the network is congested and actor–critic reinforcement learning to find better Transmission Control Protocol (TCP) parameters. We evaluated RSCAT based on three workloads: a diversified DC traffic simulation; high-performance computing from NAS benchmark; and Hadoop TeraSort application. Our results show RSCAT fundamentals (reinforcement learning, data classification, and SDN) are potential candidates to achieve zero touch network congestion control. Specifically, our experimental analysis shows RSCAT, running alongside with traditional TCP congestion control algorithms like DCTCP and CUBIC, can decrease the flow completion time and applications' runtimes in several cases, without requiring any software update on DC end-points.

1. Introduction

The network congestion phenomenon is bound to happen on contemporaneous data centers (DCs) independently of the hardware or network management policies, and solving this problem is considered a real challenge for DC administrators (Kandula et al., 2009; Noormohammadpour and Raghavendra, 2017). The congestion can happen in several ways, such as sending more packets into a server than it can handle, or too many simultaneous connections in the same network link, etc. Kandula et al. (2009). When congestion is in place, it can lead to collateral damages to the services running on the network if left untreated. Such collateral damages can vary from data losses to degraded services performances, which can then inflict other business-related problems on their own (e.g., non-compliance with service-level agreements). Given the facts, preventing congestion from happening is the first step, treating it quickly is the second (Kandula et al., 2009; Noormohammadpour and Raghavendra, 2017; La et al., 1999;

Huang et al., 2018), and a third step is to optimize the network in an autonomous way.

Essentially, two main indicators commonly perceive congestion in Transmission Control Protocol (TCP)-based networks: firstly, the network state where the network packets overload the forwarding capacity causing packet loss, even when intermediate buffers are available; and secondly, the congestion can be perceived when the packets' Round-Trip Time (RTT) passes a certain threshold and thus, the packet is empirically marked as lost and another one is sent in its place. Well-known TCP-based congestion control algorithms can be divided into two categories, local and network-assisted. Local algorithms rely solemnly on the information provided by the end-points they are running on. Without the information from the network, the algorithms have shown two main problems: they either act selfishly and try to take over the available bandwidth, or they have non-efficient decisions regarding whether or not the network is congested (La et al., 1999; Floyd

^{*} Corresponding authors.

E-mail addresses: gustavo.diel@gmail.com (G. Diel), charles.miers@udesc.br (C.C. Miers), mauricio.pillon@udesc.br (M.A. Pillon), guilherme.koslovski@udesc.br (G.P. Koslovski).

<https://doi.org/10.1016/j.jnca.2023.103639>

Received 11 November 2022; Received in revised form 11 March 2023; Accepted 2 April 2023

Available online 17 April 2023

1084-8045/© 2023 Elsevier Ltd. All rights reserved.

et al., 2004; Zhang et al., 2019; Abdelmoniem and Bensaou, 2021; Arun et al., 2022). On the other hand, network-assisted algorithms require devices support in order to perform well, but they also have configuration and optimization problems on their own (Alipio et al., 2019; Alizadeh et al., 2010; Cronkite-Ratcliff et al., 2016).

TCP relies on Receiver Window (RWND) and Congestion Window (CWND) for deciding the window size of data segments allowed to be transferred on each TCP flow. While RWND indicates the available capacity at the receiver, CWND gives insights on network congestion. Both parameters are needed due to the original congestion avoidance and control premises (Chiu and Jain, 1989): given a distributed scenario in which end-points are not aware of other end-points' demands, all the end-points sharing the same bottleneck will receive the same feedback, eventually converging to the total load on the network capacity. The premises hold even when end-points run different congestion control algorithms, as they are bounded by the same network limitation.

DCs based on Software-Defined Networking (SDN) can disrupt this scenario applying centralized congestion control and avoidance mechanisms. The SDN controller receives network data from the switches connected to it, and thus, has a comprehensive knowledge of past and ongoing flows on the network. Moreover, the controller monitors the entire network creating a robust dataset. Essentially, this data can be used to figure out which DC paths are critical to applications, and are eventually suffering from network congestion, as well as to indicate what could be the most optimal configuration for each TCP flow. Indeed, if the connection is accurately configured (e.g., slow start parameters, RWND size, and congestion control thresholds), the congesting scenario can be avoided, and the total network usage optimized.

In fact, it is possible to have thousands of servers, and even more simultaneous connections, when dealing with contemporaneous DC networks. This means that the data gathered in the controller can be voluminous, requiring the use of fast-computing algorithms to process all of it. This is where Machine Learning (ML) algorithms come in, as they will have been trained beforehand and can process large quantities of data almost in real-time due to high usage of parallel computing and mathematical optimizations (Fu et al., 2020; Bouzidi et al., 2019; Boutaba et al., 2018). Indeed what makes the ML shine in the computing world is not the composing algorithms, but the data used to train them, and that is astonishingly abundant in DC's network (Boutaba et al., 2018). Among the ML categories and paradigms, we focus on supervised data classification and Reinforcement Learning (RL), which after trained requires little computational effort to take action over continuous values (Jaakkola et al., 1994; Bhatnagar et al., 2009), composing a powerful tool-set to achieve the automation and decision-making requirements of zero touch network management (Zero touch network and Service Management (ZSM) ETSI Industry Specification Group (ISG), 2019, 2020), specifically the congestion control procedure.

In this context, we propose the Reinforcement Learning and SDN-aided Congestion Avoidance Tool (RSCAT) tool for DCs, based on the capabilities offered by SDN (managed with OpenFlow 1.3 Open Networking Foundation, 2012) to gather network data and control TCP flows.¹ We use ML techniques such as data classification to determine if the network is or is not congested, and the Actor-Critic (AC) RL method to find the appropriate RWND configuration for ongoing TCP packets, as this technique has been efficiently applied to induce congestion control (Abdelmoniem and Bensaou, 2021; Cronkite-Ratcliff et al., 2016; Moro et al., 2020) without requiring updates on legacy hosts, operating systems and modules. RSCAT is designed to act on elephant flows (e.g., backup, synchronization traffic, and large file transfers)

while not negatively impacting the mice ones (e.g., network control, and time-sensitive data).

We evaluated RSCAT by running three workloads atop a SDN-based Fat-Tree (Al-Fares et al., 2008) DC: a diversified DC traffic simulation; high-performance computing from NAS benchmark; and Hadoop TeraSort application. The results demonstrate that our RSCAT prototype, running alongside traditional end-to-end and network-assisted TCP algorithms, decreased the average Flow Completion Time (FCT) and applications' runtimes, indicating the potential application of such zero touch technique to avoid congestion on SDN-based networks.

The manuscript is organized as follows. Section 2 presents the background on ML-based DC congestion control, and SDN. Related work is presented in Section 3. Section 4 presents the design details of RSCAT. The prototype implementation and the experimental protocol are presented in Section 5, as well as the results and data analysis. Our considerations and future works are listed on Section 6.

2. Background and motivation

DC is a pool of computing and data storage resources linked together with several network devices, all working as one to run multiple services and applications with distinct needs and purposes. Under the hood, the DC network has to support heterogeneous kinds of applications, like social media, high-performance computing, data storage, and other services (Noormohammadpour and Raghavendra, 2017). In fact, the usage nature and complexity of a DC network pushes the boundaries, both physical and software-related, to their limits, and some unique issues may start appearing. One of them is the incast problem. Usually, the applications running in a DC have barrier-synchronized and many-to-one connection patterns, where multiple computers may simultaneously transmit lots of batches of data to a single aggregator node. On high loads of synchronized transmission on a single switch port, the network switch's buffers may overload, which will lead to packet loss and network performance degradation (Wu et al., 2012a; Alizadeh et al., 2010). Another problem that may arise is the outcast problem; whenever a connection is transmitting a large number of flows while another connection with fewer flows arrive at different input ports of the same switch, and are destined to the same output port, the connection with the small set of flows will have a significantly lower throughput than the larger one, out-casting them (Prakash et al., 2012). Both incast and outcast scenarios are eventually perceived and treated by the congestion control mechanisms.

2.1. Network-assisted congestion control

Due to TCP's binary-based feedback architecture, it will only change its transmission state upon segment loss or duplicated ACKs (Chiu and Jain, 1989) (or other similar control signs Huang et al., 2018; Arun et al., 2022). Whenever any loss is acknowledged in the sender, it will decrease the window value, and keep increasing while no losses occur. The congestion control works well enough in the originally proposed environment, like the Internet, but it shows its weaknesses while performing in such a heterogeneous environment like DCs. This is due to the way the regular congestion control algorithms gather their data. Usually, they only know what the sender and receiver knows, and cannot use any information from the network devices (e.g., switches, controllers) to enhance their performance. However, in DCs networks, the algorithms can take advantage of the network devices feedback.

In this sense, Explicit Congestion Notification (ECN) is an extension to TCP/IP stack to gather information from the network. When these notifications arrive, the sender may decrease the output rate, and so, preventing the congestion from worsening (Kuzmanovic et al., 2009). The network devices constantly monitor the forwarding queues, followed by the trigger of certain actions based on the device's configuration. Usually, ECN works together with a Random Early Detection (RED) technique (Wu et al., 2012b). In short, when the packet is

¹ An early version of RSCAT and preliminary results (calibration and proof-of-concept) were published at IEEE Globecom 2022 (Diel et al., 2022).

received in the network, it goes through a series of steps to check if it must be queued or gets dropped. First, the RED protocol checks if the average queue length is less than the minimum threshold; if so, the packet goes to the queue and continues normally; and if not, it checks if the average is greater than the maximum, and if so, it automatically drops the packet. If it is in between the minimum and maximum, it calculates the packet drop probability, and if it reaches a certain limit, it either gets queued or dropped (Wu et al., 2012b).

Based on both technologies (RED and ECN), the Data Center Transmission Control Protocol (DCTCP) variant aims to achieve high-bandwidth and low-latency. Moreover, DCTCP incorporates the ECN protocol to notify the servers of possible congestion, and has shown an improvement in lower latency, better buffer management, and better data burst handling than the regular TCP (Alizadeh et al., 2010). While the DCTCP does solve the incast problem in certain conditions (Alipio et al., 2019), it fails to accomplish it when dealing with a large number of concurrent servers. DCTCP also has some issues of its own, as it may also start to mark packets very early, which may cause a too-much premature indication of congestion and cause an overall decrease in throughput. DCTCP also requires a significant amount of time for a new flow to obtain its fair share of bandwidth when there is another older running flow with a large window size (Alipio et al., 2019), and as it makes a change in the TCP stack, it does require a change in the TCP server and client.

2.2. Congestion control opportunities in SDN

The SDN paradigm has become more used in the DC network bringing to the table the idea of separating the data and control planes (Kreutz et al., 2014; Open Networking Foundation, 2012). SDN creates a new central entity called controller, which takes control on the task of defining rules for the devices to follow, such as forwarding, dropping, and monitoring packets. Network devices that support SDN will connect to the controller, and will start reporting their status to it while fully detaching the control plane from themselves.

While SDN itself is not a technology that acts in any way as a congestion control algorithm, it is a paradigm that enables network developers to do so. Given that the controller has the knowledge of the entire network topology, it allows for the creation of forwarding algorithms that calculate the best possible route for each flow. Furthermore, as the controller can inject rules into SDN devices, it may as well inject a rule to intercept packets and extract their value, and thus, allowing for one more information source. It is worthwhile to mention that the capabilities SDN provides make for endless possibilities when designing a congestion control tool. The flow table also give the network administrator to apply rules on very specific connections that may be impacting the rest of the network, by using a more precise tuple to represent the flow entry. This enables the decision-making algorithm to apply actions such as lower the bandwidth or modify the packet's header information in a way that helps the overall network performance.

Due to the fact that the SDN controller has full visualization of the network, it can be used to figure out which connections are causing the network congestion, and what could possibly be the most optimal value for the TCP parameters for each connection. SDN not only can be used to calculate the ideal values, but it can also modify the TCP packets, allowing the optimization of already existing TCP congestion control and avoidance algorithms without changing anything on the server, host or the network, as this new algorithm could be a simple SDN application running alongside the controller and TCP algorithms.

Although SDN brings the tools necessary to perform such an idea, it is still hard to define the optimal parameters, once it can be reduced to a combinatorial problem. When dealing with networks, it is possible to be dealing with thousands of servers, with hundreds of thousands or even more simultaneous connections. This means that the data gathered in the controller is huge in size, and traditional algorithms might have a hard time processing such a vast amount of data. There is also the

problem of building an algorithm that is both efficient, flexible, and adaptable. It needs to be efficient so that its decision-making does not take too much time to calculate and ends up being too late to act. It must be flexible as the connection are not stable at all, as new connections are being created and destroyed all the time. And it must be adaptable as new devices can be connected or removed anytime, so fixed rules are not an ideal way to go about it. That is when the ML algorithms come in.

In short, ML algorithms work around a mathematical model that acts like an intelligence. This mathematical model is flexible and as iterations keep happening, it keeps synthesizing the outcomes and thus, learning. This process means the learning phase takes a long time to reach an optimal intelligence, but when it does reach, it starts to make decisions by only looking up into a mathematical model trained table, which is computationally fast, and, if trained properly, will return near-optimal results for the desired scenario. This means that ML algorithms are suited for large datasets, and can help SDN-based scenarios.

3. Related work

The related work presented here are examples of improving the TCP performance using various methods, even ML algorithms. Some of them use ML to classify a given connection, while others try to improve the TCP algorithm in some way. Table 1 brings a compilation of the discussed works.

3.1. Traditional DC congestion control methods

The DCTCP (Alizadeh et al., 2010) uses network devices guidance to achieve better congestion control, and while it does solve some problems and perform well under some scenarios it fails to accomplish good results when dealing with multiple concurrent servers. Even running with network support, DCTCP still makes the decisions at the end-points without full knowledge of the network traffic. In addition, DCTCP is based on the hand-tuned configuration of thresholds for managing switches queue occupancy.

In turn, in Zhang et al. (2019), the authors proposed Mystique, an automated dynamic congestion control switching scheme tool allowing the network administrator to pick the desired congestion control algorithm to be used in certain connections. It does not solve the congestion problem by itself, but instead it chooses what the best algorithm would be. Mystique requires a modification in the switches, but not in the DC end-points. This work empirically demonstrated that congestion control algorithms might be changed or reconfigured during runtime to improve overall network performance.

Regarding virtualized DCs, He et al. (2016) proposes the AC/DC TCP, a tool that enforces per-flow congestion control policies without making changes to the servers or network hardware by implementing an administration-defined algorithm over DC congestion control algorithms, like DCTCP. In turn, Cronkite-Ratcliff et al. (2016) and Abdelmoniem and Bensaou (2021) use RWND updates to induce congestion control on virtualized scenarios. While Cronkite-Ratcliff et al. (2016) intercepts packets at hypervisor layer and update RWND values before sending them to virtual machines, Abdelmoniem and Bensaou (2021) changes them to pre-calculated and optimized values with switch assistance. We explore the same technique (RWND updates) in Section 4.1 without requiring changes on DC servers.

3.2. ML-based congestion control methods

An approach to dynamically select the appropriated congestion control algorithm was proposed by Zhou et al. (2022). Instead of proposing a one-size-fits-all congestion control algorithm, the authors proposed Antelope, a system to dynamically reconfigure the network stack guided by the network state and the decisions made from a

Table 1

A compilation of related work: Yes, No, and Partial (e.g., monitor, update, solve) answers are used to identify the use of ML, modification of the TCP/IP stack, focus on congestion, changes to the hosts (servers and clients) or network switches, focus on DC networks, and finally, if they use SDN.

Ref.	ML	Modify TCP	Focus on congestion	Needs host or switch changes	DC	SDN
Alizadeh et al. (2010)	No	No	Yes	Yes	Yes	No
Abbasloo et al. (2020b)	Yes	Yes	Yes	Yes	No	No
Abbasloo et al. (2020a)	Yes	Yes	Yes	Yes	No	No
Babayigit and Ulu (2021)	Yes	No	No	No	Yes	Yes
Tosounidis et al. (2020)	Yes	No	No	No	Yes	Yes
Zhang et al. (2019)	No	Yes	Partial	Yes	Yes	Yes
Abdelmoniem and Bensaou (2021)	No	Yes	Yes	Yes	Yes	Yes
Chang et al. (2021)	Yes	No	Partial	Yes	Yes	No
Eom et al. (2021)	Yes	No	No	No	Yes	Yes
Balakiruthiga and Deepalakshmi (2021)	Yes	No	No	No	Yes	Yes
Estrada-Solano et al. (2019)	Yes	No	No	No	Yes	Yes
Fu et al. (2020)	Yes	No	No	No	Yes	Yes
Bouzidi et al. (2019)	Yes	No	No	No	Yes	Yes
He et al. (2016)	No	No	No	No	Yes	No
Ji et al. (2023)	Yes	No	Partial	No	No	No
Zhou et al. (2022)	Yes	No	Partial	Yes	Yes	No
Emara et al. (2022)	Yes	Yes	Yes	Yes	Yes	No
Xia et al. (2022)	Yes	Yes	Yes	Yes	Yes	No
RSCAT	Yes	No	Yes	No	Yes	Yes

supervised classification algorithm. The Antelope prototype was implemented in GNU/Linux, and end-points must be updated to support the dynamic selection. The results obtained with Antelope instigated our motivation for proposing a tool-set based on ML and centralized SDN knowledge to improve the overall network performance without changing the existing congestion control algorithms.

Emara et al. (2022) proposed a new congestion control, termed Pareto, based on Deep Reinforcement Learning (DRL). Similar to our approach, Pareto was developed being agnostic to hand-tuned TCP and ML parameters. However, while our proposal opts for running alongside existing TCP congestion control algorithms, Pareto is a new congestion control candidate. Similarly, Glider was proposed as a new congestion control algorithm based on DRL (Xia et al., 2022). Both approaches require updates in end-points to be executed, replacing the existing congestion control algorithms.

The project ORCA, also termed as DeepCC, is described in Abbasloo et al. (2020b,a). The authors proposed a plugin for TCP connections that aims to increase the throughput and reduce latency on cellular connections. The proposal runs at end-points and requires changes in software for all computers on the network. In turn, both Babayigit and Ulu (2021) and Tosounidis et al. (2020) present a deep learning-based load balancer for SDN networks. In addition, the MAGNet project (Chang et al., 2021) is a ML-based architecture that aims to deliver an end-point in which the devices, that support it, may send a network request and, according to the data received, will apply some processing over the network packets. This is an application-aware tool, meaning that it analyses the type and needs of each connection, and answers the best strategies to be applied in the packets.

Ji et al. (2023) demonstrated that classical congestion control strategies fail to meet the Quality of Service (QoS) requirements of contemporaneous live streaming applications, even when using Multipath TCP (MPTCP) (Iyengar et al., 2011) to improve performance. The authors proposed an adaptive QoS-aware multipath congestion control framework based on random forest regression. The framework collects selective Acknowledgments (ACKs) and computes performance metrics, which are aggregated, composing a utility function that should be maximized. Essentially, the framework gathers information directly from the TCP headers; in other words, it collects the end-host's perceptions regarding the network status without network-based support. Once the framework identifies the ideal send rate for monitoring each flow, they are adjusted at the application level.

A method for traffic classification is presented in Eom et al. (2021), in which it uses the SDN architecture to obtain network traffic data and

then runs it through a ML classification algorithm. Similar to our classification module, this is used to classify the traffic and enable future actions to be taken upon certain traffic groups. In turn, in Balakiruthiga and Deepalakshmi (2021) is proposed a traffic management tool coupled with an SDN network. This tool uses reinforcement learning to choose which and how to apply the possible configurations: controller positioning, load balancing or packets routing.

The Nelly tool is presented in Estrada-Solano et al. (2019), which uses incremental learning along with SDN to identify elephant flows. In turn, Fu et al. (2020) proposes a tool that uses Q-Learning to avoid congestion in SDN networks by applying forwarding rules, but without acting on congestion control once it is in place. Finally, Reinforcement learning is also used in Bouzidi et al. (2019) to predict traffic state and to lower the overall network latency. It also uses SDN, however only uses the quality-of-service utilities, which might not solve the congestion problem by itself.

3.3. Discussion

The review of specialized literature (summarized in Table 1) makes clear that using ML methods for solving network problems, especially congestion control, is something that has been gaining popularity. Some works deal with routing, load balancing, or forwarding of the packets, which could lead to some improvement relating to congestion avoidance and control, however it is still not enough under some situations, so another approach is needed, like changing the behavior of already existing congestion control algorithms. Another point to keep in mind is that some tools require the change of at least one of the nodes in the network, which could be a problem when the DC comprises heterogeneous connections, clients and software, so a solution that changes only network hardware or software is desirable.

In summary, our work improves the state-of-the-art as follows: (i) Instead of proposing a one-size-fits-all congestion control algorithm, we opt for improving the overall network performance by selecting appropriated TCP parameters. The rationale behind this approach is to use the human-consolidate knowledge already applied for composing workload-tailored congestion control algorithms instead of discarding them and recreating a new one, which can require a large amount of resources for training the neural network models. (ii) We propose a technique that is agnostic to end-point configurations. Essentially, the congestion control algorithms are directly reconfigured using the TCP headers via SDN devices. (iii) Our proposal uses a centralized entity to monitor the network and compose the data input for the ML-based modules. In this sense, we make decisions based on a time series of network snapshots.

4. RSCAT : concepts, modules, and design

Given the challenges and opportunities aforementioned, we propose Reinforcement Learning and SDN-aided Congestion Avoidance Tool (RSCAT), a ML-driven tool that uses SDN features to track down information and to figure out TCP parameters for each network flow. Specifically, ML is applied at two different stages running atop the SDN controller: initially, a pre-trained classification algorithm is used for detecting network status (congestion and overall load), while when it comes to deciding which TCP parameter each connection should have, an AC RL algorithm is used. The concepts, modules and design of RSCAT are detailed in the following subsections.

4.1. TCP congestion avoidance technique

Independently of the TCP algorithm used by the DC network, there are multiple parameters available to control the final performance. In this work we are interested in parameters that do not require updates on legacy hosts, operating systems, and modules so that no deep changes are required by those systems. In this sense, as the RWND management in TCP is currently implemented, it can be temporarily used to limit the CWND value of each connection, as a means of congestion control and avoidance. For each TCP flow, the maximum window size for sending data segments is defined by $\min\{CWND, RWND\}$, and consequently, by limiting RWND we implicitly limit CWND. In turn, by limiting some flows with small RWND values, the technique induces space for other flows to increase their CWND values, and consequently the throughput. This technique has been successfully applied for congestion control in virtualized scenarios (Cronkite-Ratcliff et al., 2016; Abdelmoniem and Bensaou, 2021; Moro et al., 2020). Based on that, RSCAT can manage, even so temporarily, the congestion control mechanism employed in connections of the network to help the network without hosts' modifications. In other words, RSCAT can be implemented at controller level and work together with multiple TCP variants transparently, meaning that the network is unaware of its presence. It is worth mentioning that RWND can be updated for each TCP segment, however it is required to intercept, update and re-inject some segments to reconfigure the on-going flows.

4.2. RSCAT modules

RSCAT is composed by two main modules: DC network status, and AC RL. While the first one is in charge of gathering, organizing, and detecting congestion between servers and switches, the second module is a reinforcement learning driven neural network. In short, both modules rely on SDN controller to account switches, tables, ports, and flows data. Fig. 1 illustrates the management and network data flows of the proposed mechanism. Analyzing from the bottom-up, a set of servers are interconnected by SDN-aware switches organized in a Fat-Tree topology (Al-Fares et al., 2008). All of those switches are managed by a centralized SDN controller. The controller is then running the RSCAT tool, and therefore, gathering monitored data from DC and applying TCP segments reconfiguration rules to the network. This is represented by the arrows coming in and out of the controller rectangle. Alongside the controller, RSCAT modules perform data classification and decision-making based on historical and on-going network data on the AC RL frame.

4.2.1. DC network status

This module periodically retrieves data from the SDN controller for every switch port and flow, composing snapshots of the current network state. For each switch port, RSCAT gathers the number of packets, bytes, drops, errors and collisions (received and sent), and the relative difference of received bytes between the current read event and the last one (i.e., $\frac{\text{received Bytes}}{\text{last Received Bytes}}$). The rationale behind this equation is to obtain insights about the end-points attached to a given port, in

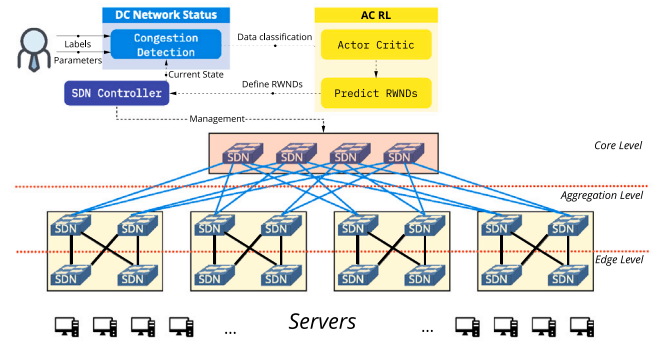


Fig. 1. General illustration of the management and network data flows.

other words, to identify whether the congestion control algorithm is increasing or decreasing its private CWND. In addition, for each flow, RSCAT accounts the number of packets and bytes transferred, as well as the maximum end-to-end bandwidth.

A supervised data classification algorithm is applied to detect on-going DC congestion, as well as to classify all flows. Based on empirical analysis, the TCP flows are classified into four main groups: high, median, and low congestion probability, or congested flows (Diel et al., 2022). The first three groups identify traffic intensity, with the last one indicating congestion. Specifically, the congested label means that something on the network has flagged a network error (such as a drop). In summary, errors, drops, collisions, and other switches and flows characteristics are combined with configuration thresholds representing the network usage (related to the total end-to-end capacity) to apply labels for each flow.

Fig. 2 depicts the flows labels and their characteristics. Initially, all flows are classified in the high usage group. Latter, previously monitored flows are accounted as congested, median, and low, if necessary. Otherwise remains with the high label. As indicated by Fig. 2(a), a flow is classified as congested in the occurrence of errors, drops or collision events. These events come directly from the network through SDN monitored data. The other flows are classified according to the usage of the total available bandwidth for a given path (Fig. 2(b)). Summarizing, flows with less than 40% of usage are classified as low usage, between 40% and 70% as median while everything else is classified as high usage flows (the thresholds were empirically defined and can be refined, if need). The rationale behind this approach is to represent the impact of private TCP decisions regarding the increasing or decreasing of the segment's window. Upon finishing this classification, the AC RL module is invoked, receiving as input the same network data used to classify along with the classification results.

4.2.2. AC RL to configure TCP flows

Following the data flow depicted in Fig. 1, the decision process starts with the execution of the DC network data classifier, which invokes the AC RL module. Among the reinforcement learning techniques, the AC method is a model-free Temporal Difference Learning (TDL) method (Sutton, 1988) that employs two distinct structures to represent the policy table independently of the value function. The policy structure is known as the actor, as it is the one to take action, while the value function is the critic, as it criticizes the actions taken by the actor, which comes out as the reward of each action.

Internally, the actor is responsible for using the base set of rules to learn how to respond to each scenario, receiving various inputs that it has never seen before and trying to get the appropriated answer. The actor selects the appropriate RWND for each connection, from a set of predefined options: 4 KB, 128 KB, 1 MB, 8 MB and 32 MB. Those values were chosen by selecting the most common default values in GNU/Linux-based systems for the default RWND, and adding some

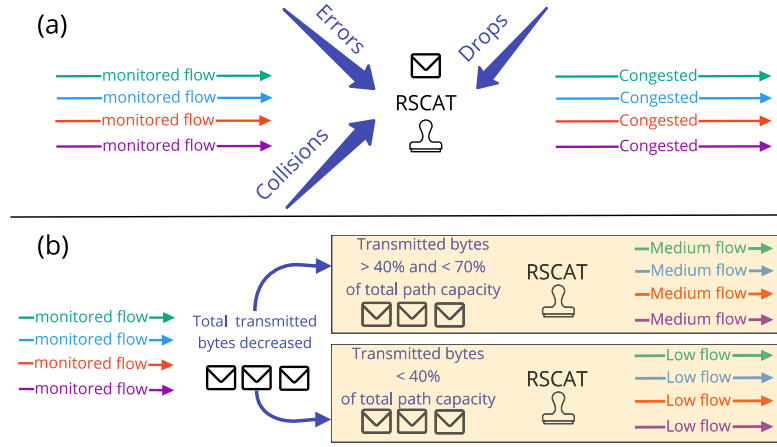


Fig. 2. Classification labels and action thresholds.

steps in-between. RSCAT uses AC RL to individually analyze the ongoing TCP flows. Each time it is invoked, the AC RL module runs through a list of TCP flows asking the actor which configuration should be used. Subsequently, the set of actions taken by the actor is evaluated by the critic. Specifically, the critic judges them following predetermined base rules, current state and the previous one, giving the actor a score for its last action and learning how to judge better.

The AC aims at maximizing the accumulated reward, which is account as depicted by Algorithm 1. Specifically, *eql_threshold* is used to determine if two throughputs are similar (it has an empirically defined value of 5%). For each flow, the algorithm analyzes the current throughput achieved from the latest RWND intervention, choosing a return value between $\{-10, -5, -2, 5, 10\}$ guided by the data classification thresholds and network status evolution. The actor then uses this value to learn how well it has performed. It is important to note that in the following runs, the critic will also receive the difference of states as its input, so that it can also learn how to perform better. In the long-term, the training will then be near optional, and both modules can then start to run in production mode. At this point, the AC RL training and validation phases are no longer needed and can be disabled to improve the overall performance.

Finally, following the dataflow from Fig. 1, the SDN controller is informed about all TCP flows that must be reconfigured and their correspondent RWND configurations. At this stage, RSCAT can then intercept some TCP segments and reconfigure them with the new optimal RWND value so that the network end-points can use this value internally.

Algorithm 1: AC reward computation.

```

1 if current_throughput - last_throughput < eql_threshold then
2   if last action was to lower RWND then
3     reward is good(5)
4   else if last action was to increase RWND then
5     reward is somewhat bad(-2)
6   end
7 else if current_throughput < last_throughput then
8   if last action was to lower RWND then
9     reward is very bad(-10)
10  else if last action was to increase RWND then
11    reward is very good(10)
12  end
13 else
14   if last action was to lower RWND then
15     reward is bad(-5)
16   else if last action was to increase RWND then
17     reward is good(5)
18   end
19 end

```

4.3. Example and details

We present a simple use-case scenario to exemplify the use of RSCAT and show how each step and ML technique produce a corresponding result. Lets use Fig. 1 as the baseline for discussion. The DC Network Status module periodically asks the SDN controller to retrieve a network snapshot. This data summarizes the network status based on switches and port metrics directly offered by the OpenFlow protocol. A dataset is composed by aggregating such information, as represented by Table 2. The TCP data between two end-points can flow between multiple switches (composing a path). Each row represents a single switch perspective about a TCP flow. All the data comes directly from the network, except the columns Byte Diff, Packet Diff, and Relative Diff, calculated with the data from prior snapshots, indicating the network evolution. A supervised data classification algorithm is applied to detect the ongoing DC congestion, classifying each flow as High, Medium, and Low congestion probability, or Congested flows (as depicted by Fig. 2).

Following the data flow depicted in Fig. 1, the decision process starts with the execution of the AC RL module. Table 3 demonstrate an example of a dataset received by the AC RL module for making the reconfiguration decisions. The actor observes the network dataset (the original version with extended classification regarding the congestion probability) to make decisions. For each ongoing flow, the actor defines if the TCP should improve or decrease the number of segments flowing through the network. Moreover, the actor decides the size of RWND for the next rounds.

In this sense, for each flow (in other words, for each actor's decision), the critic observes if the action increased or decreased the overall network performance. The critic composes the reward by observing the current and last network throughputs, as indicated by the Algorithm 1. In the long-term, the objective is to maximize the AC reward, consequently improving the overall network performance. Finally, it is worthwhile to mention that AC RL actions tend to improve the network performance of elephant flows, as the reconfigurations are not immediately applied. We discussed this behavior and other perspectives in Section 5.

5. Implementation and validation

This section discusses the prototype implementation of RSCAT, followed by the experimental protocol, results, and key observations.

5.1. Prototype implementation

As a proof-of-concept, RSCAT was implemented and tested based on Mininet (Lantz et al., 2010), Ryu controller (Ryu S.D.N. Framework Community, 2017), OpenFlow protocol version 1.3, and Python language. Details are provided in the following subsections.

Table 2

Example of data gathered by the DC Network Status module.

Byte Diff	Packet Diff	RX packets	TX packets	RX bytes	TX bytes	RX dropped	TX dropped	RX errors	TX errors	Collisions	Relative Diff
1088527845	19483	9635	11096	446610368	756324	0	0	0	0	0	0.03
1607960	23832	15577	16002	454105510	302974162	0	0	0	0	0	-0.03
0	0	20511	21534	580593326	329119969	0	0	0	0	0	-0.02
0	0	16048	15617	328203743	449025846	0	0	0	0	0	0.06
1780822	26455	15736	15134	303251548	433681424	0	0	0	0	0	0.1
4222022	61075	11100	9166	748104	597796356	0	0	0	0	0	-0.07
3177492	46942	15131	15738	433681226	303251688	1	0	0	0	2	0.16
573073451	3682	10168	11498	467205964	782928	0	0	0	0	1	-0.05

Table 3

Example of data input received by the AC RL module.

Byte Diff	Packet Diff	RX packets	TX packets	RX bytes	TX bytes	RX dropped	TX dropped	RX errors	TX errors	Collisions	Relative Diff	State
1780822	26455	15736	15134	303251548	433681424	0	0	0	0	0	0.1	Low
4222022	61075	11100	9166	748104	597796356	0	0	0	0	0	-0.07	High
3177492	46942	15131	15738	433681226	303251688	1	0	0	0	2	0.16	Congested
573073451	3682	10168	11498	467205964	782928	0	0	0	0	1	-0.05	Medium

5.1.1. Data monitoring and packets update

The Ryu controller offers an extensive SDN toolset adherent to OpenFlow standard. For example, it allows the developer to define some callback methods that correspond to different network activities. Specifically, the OpenFlow-defined events *EventOFFPStateChange*, *EventOFFPFlowStatsReply* and *EventOFFPPortStatsReply* were used to implement the prototype. The former listens for any change in the network, like adding new network devices, which was used to generate an in-memory virtual topology of the network. The *EventOFFPFlowStatsReply* event captures the transmission of any packet from a given flow. This event is the most used one: for every packet we extract the data and write into a file for further analysis. For the sake of performance, this file writing action only happens every 5 seconds (the parameter can be redefined, if need). Finally, the *EventOFFPPortStatsReply* follows a similar rationale, however it happens for every packet from a given port.

Considering the Ryu controller and the OpenFlow standard, there are two ways for overwriting the RWND value on the TCP headers. First using the *OFPTFPT_WRITE_SETFIELD* action to make the switches write into the header, while another way, is to drop the original packet, and use the controller send message calls to make the switches send a new TCP packet with the exact same data, except for the RWND. The proof-of-concept RSCAT prototype used the second technique.

5.1.2. Learning modules and neural networks

The data classification module (DC network status, from Fig. 1) used the algorithms provided by the Python library SKLearn (Pedregosa et al., 2011). Specifically, Random Forest (RF), Support Vector Machine (SVM), and Multilayer Perceptron (MLP) were imported from the library. The calibration analysis (Diel et al., 2022) indicated that RF outperformed its counterparts, being adopted as the default choice for the prototype. The second main module of RSCAT (AC RL from Fig. 1) was implemented in Python language based on PyTorch framework (Paszke et al., 2019). Both Actor and Critic follow the same base composition of 3 layers, 596 nodes, 12 inputs and 1 output, resulting in 17408 node connections. While both have the same composition, their memory space is different and one does not interfere with the other.

Regarding the AC RL neural networks, the Adam optimizer was selected as empirical results demonstrated its outperforms other stochastic methods (Kingma and Ba, 2014). For our prototype, Adam optimizer was configured using a learning rate of $1e-5$, $\gamma = 0.99$, and a 32 epochs sized memory. Our approach of AC is constantly learning, and in this sense, there is no predefined limit for the number of running epochs. Finally, the gradient loss method was used by both, actor and critic networks.

5.2. Experimental protocol and infrastructure

The experimental protocol and the results obtained by running RSCAT alongside two popular variants of TCP congestion control algorithms, DCTCP and CUBIC, are presented in this section. In short, three experimental scenarios are discussed:

- Section 5.3 investigates the performance of RSCAT when executing a DC-like workload generated by the Empirical Traffic Generator (ETG) simulator developed by Cisco (Cisco, 2019). Multiple pairs of ETG client-servers were instantiated to execute the ETG input configuration proposed by Mentz et al. (2020), which summarizes the DC traffic described in Benson et al. (2010).
- Section 5.4 contains the results and discussion about an High Performance Computing (HPC) DC. For representing the requirements of multiple HPC applications, this scenario employs the NAS Parallel Benchmark (NPB) (Bailey et al., 1991). The benchmark is composed of applications with distinct communication requirements (Marcondes et al., 2016). Specifically, we executed the class B configuration of NPB, and the following applications were selected Block Tri-diagonal solver (BT), Conjugate Gradient (CG), Embarrassingly Parallel (EP), Discrete 3D Fast Fourier Transform (FT), Lower-Upper Gauss-Seidel solver (LU), Multi-Grid solver (MG) and Scalar Penta-diagonal solver (SP), all using the OpenMPI (Gabriel et al., 2004) version 4.0.3.
- Finally, Section 5.5 details the execution of the large-scale Apache TeraSort Benchmark (Huang et al., 2010). In short, TeraSort is a standard map/reduce sort application for Hadoop traditionally used as benchmark.

The scenarios are representative for demonstrating the performance (benefits and drawbacks) of RSCAT with regarding distinct DC traffic workloads and characteristics, addressing the challenges and opportunities discussed in Section 2. For instance, the Apache TeraSort scenario is representative for the incast traffic challenge, while the ETG is capable of generating traffic with combined incast and outcast situations. In turn, the applications composing the NPB scenario have elephants and mice flows combined, imposing a trick classification and improvement scenario. The common elements of the experimental protocol are explained in Section 5.2, and latter the scenarios are individually analyzed (Sections 5.3–5.5), followed by a key observations discussion (Section 5.6).

For composing the experimental scenarios, we selected Mininet version 2.3.0 (Lantz et al., 2010), and created a DC interconnected by a Fat-Tree topology (Al-Fares et al., 2008). The Fat-Tree topology is demonstrated in Fig. 1, configured with $k = 4$, meaning that there are 4 core switches, connected to 4 pods (each containing 2 aggregation

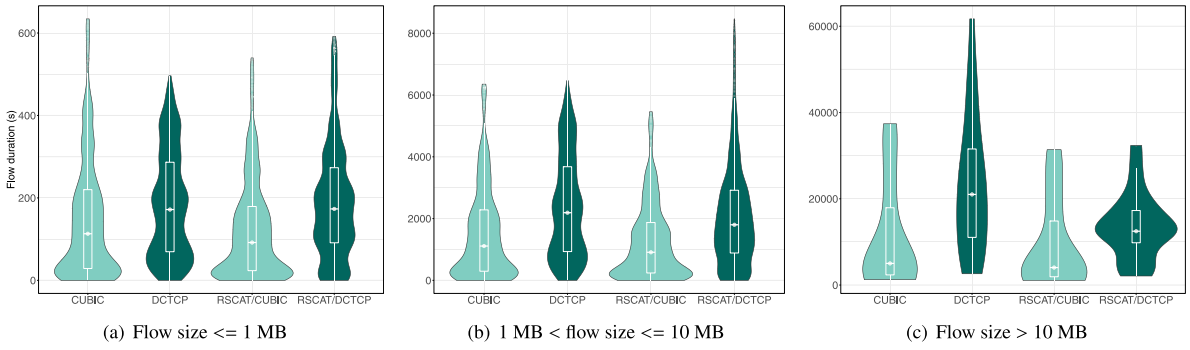


Fig. 3. Time (in seconds) for the execution of ETG workload.

switches and 2 edge switches each) with all of the edge switches connected to servers. Each link supports 1 Gbps, and the OpenFlow version 1.3 (Open Networking Foundation, 2012) was selected as the south-bound interface of SDN topology, managed by the Ryu controller (Ryu S.D.N. Framework Community, 2017). For every single experiment and run, the controller was restarted, Mininet's cache was cleared and no application was running in the background, only the core operating system processes.

Finally, it is worth mentioning that the whole infrastructure was hosted on a single computer, 8-cores, 16-threads i9 CPU with 48 GB of RAM, running Ubuntu 20.04. During runs, the average controller's CPU usage was hovering over 5%, with spikes up to 20% when new flows were initiated. RAM usage was also a non-issue, as it was approximately 336 MB during every experiment.

5.2.1. Metrics

We selected two metrics for representing the performance of RSCAT: execution time and Flow Completion Time (FCT). The execution time (applicable for NPB and Apache TeraSort scenarios) represents the high-level users' perspective, while the FCT gives insights on the internals of RSCAT performance, demonstrating, for instance, the benefits and drawbacks when dealing with elephants and mice traffic patterns (Dukkipati and McKeown, 2006).

5.2.2. Comparisons

The experiments compare the algorithms CUBIC and DCTCP, with and without the RSCAT support. It is important to highlight (details were discussed in Section 4) that RSCAT runs alongside with traditional TCP congestion control algorithms. In this sense, the goal is not to replace the TCP congestion control algorithm, on the other hand, we aim to demonstrate that RSCAT can be combined with traditional algorithms to improve the overall DC's performance following a zero touch management methodology. Initially, the CUBIC and DCTCP were individually executed, followed by jointly executions with RSCAT. Each scenario was repeated 10 times.

5.3. Empirical Traffic Generator (ETG) scenario

The results for the Empirical Traffic Generator (ETG)-driven scenario are summarized in Fig. 3. We organized the data according to flow size intervals to differentiate the RSCAT performance with mice and elephant flows: sizes smaller than 1 MB, between 1 MB and 10 MB, and greater than 10 MB. For this experiment, we discuss the FCT metric (see Section 5.2), presented as violin plots to show the entire distribution of the data besides the upper, lower, and quartiles points.

Initially, the results indicate that RSCAT was unable to decrease the FCT for mice flows (Fig. 3(a)), independently of the baseline CUBIC or DCTCP. This behavior was expected, as the RSCAT did not have time to make a correct decision. As the flow sizes increased, the scenario changed considerably, as denoted by Figs. 3(b) and 3(c). Specifically analyzing the median value for flow sizes greater than

10 MB (Fig. 3(c)), RSCAT decreased almost 40% and 19% for DCTCP and CUBIC, respectively. Elephant flows take advantage from RSCAT once they overcame the traditional congestion control limits, improving the overall throughput. Finally, the data distribution (graphically summarized by the area shapes on Fig. 3) demonstrate that RSCAT concentrates the values close to the minimum value, and in the worst case, close to the medium values.

5.4. NAS Parallel Benchmark scenario

The NAS Parallel Benchmark (NPB) is a powerful benchmark representing HPC applications (Bailey et al., 1991) with distinct networking requirements in terms of communication topology, latency, and throughput (Marcondes et al., 2016). For composing the experimental scenario, each application ran individually, and to avoid any possible caching interference, we first ran all of the algorithms before starting to repeat the run. The NPB results are illustrated as 7 boxplots in Fig. 4, one for each selected application. Each boxplot diagram represents the minimum, maximum, median, first quartile, and third quartile values. The black dots represent values outside the boundaries.

Before discussing the results, it is important to understand what each algorithm is doing behind the scenes, and consequently, what we can expect in terms of improvements. EP is the algorithm that has minimal network usage, so RSCAT could not improve its performance as there was no heavy network traffic to enhance (Marcondes et al., 2016). Even though, in this case, RSCAT CUBIC reached a few seconds less than CUBIC. A similar effect happens to FT, which has only mice flows to synchronize the execution. This behavior can be seen on Figs. 4(c) and 4(d) where the medians are close to each other. MG, LU, CG, SP and BT are algorithms with higher network usage. Each one has its own communication pattern (i.e., ring topology, sequential communication, synchronization metrics), and specifically, SP transfers the highest data volume (elephants flows), while the others are composed of small-scale flows (Marcondes et al., 2016). When analyzing SP (Fig. 4(g)), RSCAT with CUBIC has a stable behavior and improves its overall performance, while RSCAT with DCTCP achieve the minimum value.

In short, the NPB algorithms with higher network usage have shown to benefit from the usage of RSCAT. The performance improvement is perceived in all boxplots, in which RSCAT has a lower median time (the black line in the middle of the rectangles), and usually a lower minimal point. This scenario has shown that RSCAT takes advantage of long-running traffic, where small flows have little to no advantages, but longer-running algorithms could increase their performance and execute in lower time compared to when not running RSCAT. Finally, in all cases RSCAT had the shortest time, either with RSCAT CUBIC or RSCAT DCTCP.

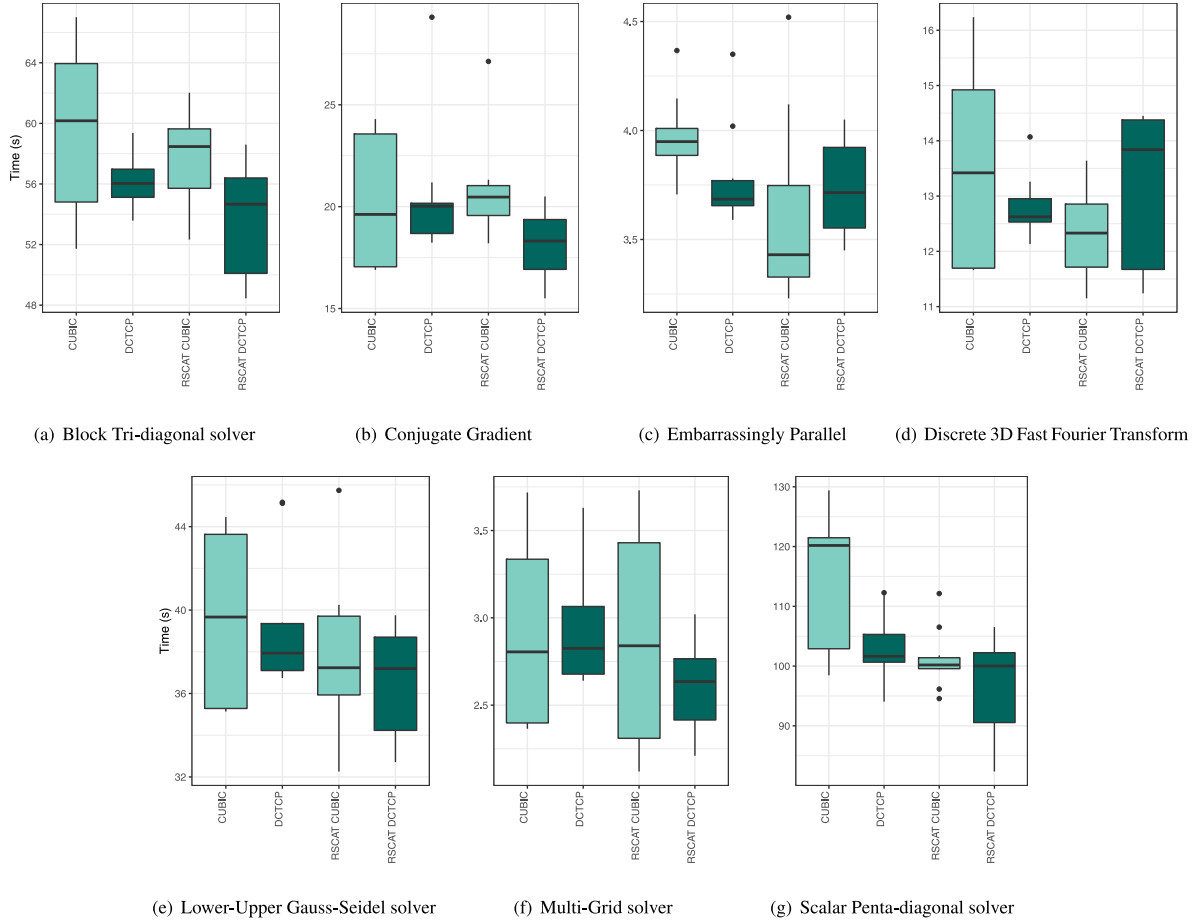


Fig. 4. Time (in seconds) for the execution of NPB applications.

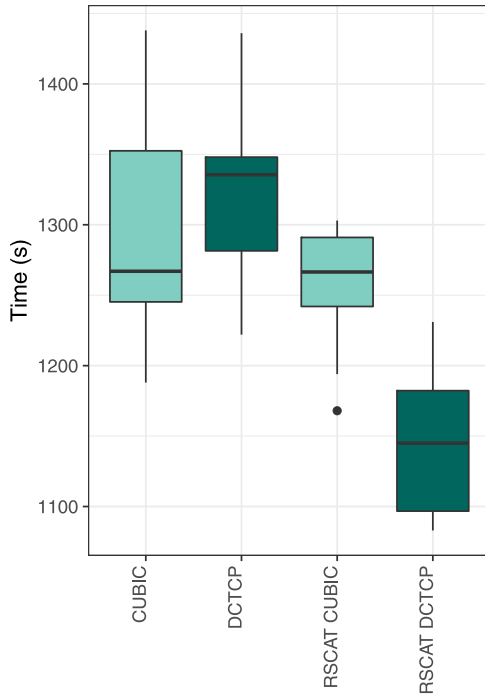


Fig. 5. Time (in seconds) for the TeraSort execution.

5.5. Apache Hadoop TeraSort scenario

This scenario is based on the TeraSort benchmark, which process a large-scale data set with the Hadoop platform (Hadoop Distributed File System (HDFS) and MapReduce) (Dean and Ghemawat, 2008). In essence, the MapReduce model is coordinated by few control messages. RSCAT is unable to optimize the small control flows, however can benefit the overall MapReduce computation phases, representing the incast problem discussed in Section 2. The results are summarized in Fig. 5. Although the CUBIC and RSCAT with CUBIC medians are similar, only the RSCAT CUBIC results data are stable (the first and third quartiles are close). Finally, the results of RSCAT with DCTCP deserve to be highlighted. First, the DCTCP minimum time is close to the DCTCP with RSCAT maximum time, and moreover, comparing the medians, RSCAT decreases the execution time by 25% compared with DCTCP, demonstrating a clear overall improvement offered by RSCAT.

5.6. Key observations and discussion

The experimental discussion indicates the benefits and drawbacks facing three distinct workloads. Independently of the workload, the DC's administrator would not want to set TCP parameters and thresholds manually, nor worry about network congestion and quality-of-service of each individual application. In this sense, RSCAT is a tool proposed to run alongside with existing DC congestion avoidance and control protocols. From a macro analysis on results, it is evident RSCAT was unable to improve small-scale mice flows, keeping the same results from original algorithms in most cases (Section 5.3). This behavior is justified by the RSCAT inability to discover issues and optimize the DC switches in acceptable time. However, RSCAT successfully improved

elephant-flows based scenarios. For scenarios with dedicated executions and well-defined communication patterns (such as HPC workload, Section 5.4), RSCAT improved the overall performance. The same behavior was identified for the TeraSort workload (Section 5.5). In summary, the scenarios demonstrated how zero-touch network congestion control can improve the performance of DC hosted applications running different workloads.

6. Considerations and future work

The existence of congestion in DC networks is a quality-of-service threat to every application. Dealing with this threat is necessary, however it is a hard challenge. Recently, SDN offered a new opportunity to gather network information in a logic and centralized manner. Given this opportunity, this work proposed RSCAT, a tool that uses information available in SDN controllers to enhance current existing TCP congestion control algorithms based on ML techniques. RSCAT makes use of ML algorithms to predict the best possible RWND size for each connection, so that the SDN controller may alter the TCP packet header to update the RWND value. This change will temporally induce the internal end point control, helping to avoid the congestion.

The results demonstrated that when applying RSCAT along with existing congestion control algorithms like CUBIC or DCTCP, the overall network performance has increased. The experimental protocol used three distinct workloads to highlight the advantages and drawbacks of RSCAT. Specifically, when dealing with elephant traffic, RSCAT has shown great results. Summing up, RSCAT demonstrated how a zero-touch congestion control can be achieved on DC networks. As future work, we aim at improving the RSCAT's dataset with information from switches queues, as well as investigate other reward functions and neural network's parameters for the ML modules.

CRediT authorship contribution statement

Gustavo Diel: Wrote the main manuscript text, Designed and developed the RSCAT prototype, Executed the experimental scenario, Analyzed the experimental data and approved the submitted version. **Charles Christian Miers:** Wrote the main manuscript text, Analyzed the experimental data, Approved the submitted version. **Maurício Aronne Pillon:** Wrote the main manuscript text, Analyzed the experimental data, Elaborated graphs, Approved the submitted version. **Guilherme Piêgas Koslovski:** Wrote the main manuscript text, Designed the RSCAT, Elaborated the experimental protocol, Analyzed the experimental data, Approved the submitted version.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request

Acknowledgments

This work was funding by the National Council for Scientific and Technological Development (CNPq), Brazil, Santa Catarina State Research and Innovation Support Foundation (FAPESC), Brazil, UDESC, Brazil, and developed at LabP2D.

References

- Abbasloo, S., Yen, C.-Y., Chao, H.J., 2020a. Classic meets modern: A pragmatic learning-based congestion control for the internet. In: Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication on the Applications, Technologies, Architectures, and Protocols for Computer Communication. pp. 632–647.
- Abbasloo, S., Yen, C.-Y., Chao, H.J., 2020b. Wanna make your TCP scheme great for cellular networks? Let machines do it for you!. *IEEE J. Sel. Areas Commun.* 39 (1), 265–279.
- Abdelmoniem, A.M., Bensaou, B., 2021. Implementation and evaluation of data center congestion controller with switch assistance. *arXiv preprint arXiv:2106.14100*.
- Al-Fares, M., Loukissas, A., Vahdat, A., 2008. A scalable, commodity data center network architecture. *ACM SIGCOMM Comput. Commun. Rev.* 38 (4), 63–74.
- Alipio, M., Tiglaio, N.M., Bokhari, F., Khalid, S., 2019. TCP incast solutions in data center networks: A classification and survey. *J. Netw. Comput. Appl.* 146, 102421.
- Alizadeh, M., Greenberg, A., Maltz, D.A., Padhye, J., Patel, P., Prabhakar, B., Sengupta, S., Sridharan, M., 2010. Data center TCP (DCTCP). In: Proceedings of the ACM SIGCOMM 2010 Conference. pp. 63–74.
- Arun, V., Alizadeh, M., Balakrishnan, H., 2022. Starvation in end-to-end congestion control. In: Proceedings of the ACM SIGCOMM 2022 Conference. SIGCOMM '22, Association for Computing Machinery, New York, NY, USA, pp. 177–192.
- Babayigit, B., Ulu, B., 2021. Deep learning for load balancing of SDN-based data center networks. *Int. J. Commun. Syst.* 34 (7), e4760.
- Bailey, D., Barszcz, E., Barton, J., Browning, D., Carter, R., Dagum, L., Fatoohi, R., Frederickson, P., Lasinski, T., Schreiber, R., Simon, H., Venkatakrishnan, V., Weeratunga, S., 1991. The nas parallel benchmarks. *Int. J. High Perform. Comput. Appl.* 5 (3), 63–73.
- Balakiruthiga, B., Deepalakshmi, P., 2021. (ITMP)—Intelligent traffic management prototype using reinforcement learning approach for software defined data center (SDDC). *Sustain. Comput. Inform. Syst.* 100610.
- Benson, T., Akella, A., Maltz, D.A., 2010. Network traffic characteristics of data centers in the wild. In: Proceedings of the 10th ACM SIGCOMM Conference on Internet Measurement. IMC '10, pp. 267–280.
- Bhatnagar, S., Sutton, R.S., Ghavamzadeh, M., Lee, M., 2009. Natural actor-critic algorithms. *Automatica* 45 (11), 2471–2482.
- Boutaba, R., Salahuddin, M.A., Limam, N., Ayoubi, S., Shahriar, N., Estrada-Solano, F., Caicedo, O.M., 2018. A comprehensive survey on machine learning for networking: evolution, applications and research opportunities. *J. Internet Serv. Appl.* 9 (1), 1–99.
- Bouzi, E.H., Outtagarts, A., Langar, R., 2019. Deep reinforcement learning application for network latency management in software defined networks. In: 2019 IEEE Global Communications Conference. GLOBECOM, IEEE, pp. 1–6.
- Chang, H., Kodialam, M., Lakshman, T., Mukherjee, S., Van der Merwe, J.K., Zaheer, Z., 2021. MAGNET: Machine learning guided application-aware networking for data centers. *IEEE Trans. Cloud Comput.*
- Chiu, D.-M., Jain, R., 1989. Analysis of the increase and decrease algorithms for congestion avoidance in computer networks. *Comput. Netw. ISDN Syst.* 17 (1), 1–14.
- CISCO, 2019. Datacenter empirical traffic gen: Simple client-server application for generating user-defined traffic patterns. URL <https://github.com/datacenter/empirical-traffic-gen>.
- Cronkite-Ratcliff, B., Bergman, A., Vargafik, S., Ravi, M., McKeown, N., Abraham, I., Keslassy, I., 2016. Virtualized congestion control. In: Proceedings of the 2016 ACM SIGCOMM Conference. pp. 230–243.
- Dean, J., Ghemawat, S., 2008. MapReduce: simplified data processing on large clusters. *Commun. ACM* 51 (1), 107–113.
- Diel, G., Miers, C.C., Pillon, M., Koslovski, G., 2022. Data classification and reinforcement learning to avoid congestion on SDN-based data centers. In: 2022 IEEE Global Communications Conference: Next-Generation Networking and Internet. Globecom 2022 NGNI, Rio de Janeiro, Brazil.
- Dukkipati, N., McKeown, N., 2006. Why flow-completion time is the right metric for congestion control. *ACM SIGCOMM Comput. Commun. Rev.* 36 (1), 59–62.
- Emara, S., Wang, F., Li, B., Zeyl, T., 2022. Pareto: Fair congestion control with online reinforcement learning. *IEEE Trans. Netw. Sci. Eng.* 9 (5), 3731–3748.
- Eom, W.-J., Song, Y.-J., Park, C.-H., Kim, J.-K., Kim, G.-H., Cho, Y.-Z., 2021. Network traffic classification using ensemble learning in software-defined networks. In: 2021 International Conference on Artificial Intelligence in Information and Communication. ICAIIC, IEEE, pp. 089–092.
- Estrada-Solano, F., Caicedo, O.M., Da Fonseca, N.L., 2019. Nelly: Flow detection using incremental learning at the server side of sdn-based data centers. *IEEE Trans. Ind. Inform.* 16 (2), 1362–1372.
- Floyd, S., Henderson, T., Gurtov, A., 2004. RFC3782: The newreno modification to tcp's fast recovery algorithm. RFC Editor.
- Fu, Q., Sun, E., Meng, K., Li, M., Zhang, Y., 2020. Deep Q-learning for routing schemes in SDN-based data center networks. *IEEE Access* 8, 103491–103499.
- Gabriel, E., Fagg, G.E., Bosilca, G., Angskun, T., Dongarra, J.J., Squyres, J.M., Sahay, V., Kambadur, P., Barrett, B., Lumsdaine, A., et al., 2004. Open MPI: Goals, concept, and design of a next generation MPI implementation. In: European Parallel Virtual Machine/Message Passing Interface Users' Group Meeting. Springer, pp. 97–104.

- He, K., Rozner, E., Agarwal, K., Gu, Y., Felter, W., Carter, J., Akella, A., 2016. AC/DC TCP: Virtual congestion control enforcement for datacenter networks. In: Proceedings of the 2016 ACM SIGCOMM Conference. pp. 244–257.
- Huang, S., Dong, D., Bai, W., 2018. Congestion control in high-speed lossless data center networks: A survey. *Future Gener. Comput. Syst.* 89, 360–374.
- Huang, S., Huang, J., Dai, J., Xie, T., Huang, B., 2010. The HiBench benchmark suite: Characterization of the MapReduce-based data analysis. In: 2010 IEEE 26th International Conference on Data Engineering Workshops. ICDEW 2010, pp. 41–51.
- Iyengar, J., Raiciu, C., Barre, S., Handley, M.J., Ford, A., 2011. Architectural guidelines for multipath TCP development. (6182), Request for Comments, RFC Editor.
- Jaakkola, T., Jordan, M.I., Singh, S.P., 1994. On the convergence of stochastic iterative dynamic programming algorithms. *Neural Comput.* 6 (6), 1185–1201.
- Ji, X., Han, B., Xu, C., Song, C., Su, J., 2023. Adaptive QoS-aware multipath congestion control for live streaming. *Comput. Netw.* 220, 109470.
- Kandula, S., Sengupta, S., Greenberg, A., Patel, P., Chaiken, R., 2009. The nature of data center traffic: measurements & analysis. In: Proceedings of the 9th ACM SIGCOMM Conference on Internet Measurement. pp. 202–208.
- Kingma, D.P., Ba, J., 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*.
- Kreutz, D., Ramos, F.M., Verissimo, P.E., Rothenberg, C.E., Azodolmolky, S., Uhlig, S., 2014. Software-defined networking: A comprehensive survey. *Proc. IEEE* 103 (1), 14–76.
- Kuzmanovic, A., Mondal, A., Floyd, S., Ramakrishnan, K., 2009. Adding explicit congestion notification (ECN) capability to TCP's SYN/ACK packets. RFC5562.
- La, R.J., Walrand, J., Anantharam, V., 1999. Issues in TCP Vegas. Citeseer.
- Lantz, B., Heller, B., McKeown, N., 2010. A network in a laptop: rapid prototyping for software-defined networks. In: Proceedings of the 9th ACM SIGCOMM Workshop on Hot Topics in Networks. pp. 1–6.
- Marcondes, A.H., Diel, G., de Souza, F.R., Vieira, P.R., Fiorese, A., Koslovski, G.P., 2016. Executing distributed applications on sdn-based data center: A study with nas parallel benchmark. In: 2016 7th International Conference on the Network of the Future. NOF, IEEE, pp. 1–3.
- Mentz, L.L., Loch, W.J., Koslovski, G.P., 2020. Comparative experimental analysis of docker container networking drivers. In: IEEE Int. Conference on Cloud Networking. CloudNet, pp. 1–7.
- Moro, V., Pillon, M.A., Miers, C.C., Koslovski, G.P., 2020. Analysis of virtualized congestion control in applications based on hadoop MapReduce. In: Bianchini, C., Osthoff, C., Souza, P., Ferreira, R. (Eds.), *High Performance Computing Systems*. Springer International Publishing, Cham, pp. 37–52.
- Noormohammadpour, M., Raghavendra, C.S., 2017. Datacenter traffic control: Understanding techniques and tradeoffs. *IEEE Commun. Surv. Tutor.* 20 (2), 1492–1525.
- Open Networking Foundation, 2012. OpenFlow Version 1.3.0. Open Networking Foundation.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., Chintala, S., 2019. PyTorch: An imperative style, high-performance deep learning library. In: *Advances in Neural Information Processing Systems 32*. Curran Associates, Inc., pp. 8024–8035.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., Duchesnay, E., 2011. Scikit-learn: Machine learning in python. *J. Mach. Learn. Res.* 12, 2825–2830.
- Prakash, P., Dixit, A., Hu, Y.C., Kompella, R., 2012. The {tcp} outcast problem: Exposing unfairness in data center networks. In: 9th {USENIX} Symposium on Networked Systems Design and Implementation. NSDI 12, pp. 413–426.
- Ryu S.D.N. Framework Community, 2017. Ryu SDN controller. URL <https://ryu-sdn.org/>.
- Sutton, R.S., 1988. Learning to predict by the methods of temporal differences. *Mach. Learn.* 3 (1), 9–44.
- Tosounidis, V., Pavlidis, G., Sakellariou, I., 2020. Deep Q-learning for load balancing traffic in SDN networks. In: 11th Hellenic Conference on Artificial Intelligence. pp. 135–143.
- Wu, H., Feng, Z., Guo, C., Zhang, Y., 2012a. ICTCP: Incast congestion control for TCP in data-center networks. *IEEE/ACM Trans. Netw.* 21 (2), 345–358.
- Wu, H., Ju, J., Lu, G., Guo, C., Xiong, Y., Zhang, Y., 2012b. Tuning ECN for data center networks. In: Proceedings of the 8th International Conference on Emerging Networking Experiments and Technologies. pp. 25–36.
- Xia, Z., Wu, L., Wang, F., Liao, X., Hu, H., Wu, J., Wu, D., 2022. Glider: Rethinking congestion control with deep reinforcement learning. *World Wide Web* 26 (1), 115–137.
- Zero touch network and Service Management (ZSM) ETSI Industry Specification Group (ISG), 2019. Zero-touch Network and Service Management (ZSM); Requirements Based on Documented Scenarios. Tech. rep., European Telecommunications Standards Institute (ETSI).
- Zero touch network and Service Management (ZSM) ETSI Industry Specification Group (ISG), 2020. Zero-touch network and service management (ZSM); means of automation. Tech. rep., European Telecommunications Standards Institute (ETSI).
- Zhang, Y., Cui, L., Tso, F.P., Guan, Q., Jia, W., Zhou, J., 2019. Mystique: a fine-grained and transparent congestion control enforcement scheme. *IEEE Trans. Netw. Serv. Manag.* 16 (4), 1869–1883.
- Zhou, J., Qiu, X., Li, Z., Li, Q., Tyson, G., Duan, J., Wang, Y., Pan, H., Wu, Q., 2022. A machine learning-based framework for dynamic selection of congestion control algorithms. *IEEE/ACM Trans. Netw.* 1–16.

Gustavo Diel He received his master's degree in applied computing and bachelor's degree in computer science from Santa Catarina State University (UDESC) in Joinville/SC – Brazil. He carried out research activities related to Software Defined Networking, congestion control/avoidance in large-scale DCs, and routing at LabP2D (Laboratory of Parallel and Distributed Processing) of UDESC. Currently, he is working as a back-end and DevOps engineer.

Charles Christian Miers Professor of Computer Networks Security, and Computer Networks at Santa Catarina State University (UDESC) in Joinville/SC. He received his doctorate from Polytechnic School of the University of São Paulo (EPUSP) in Computer Engineering, the master's degree from Federal University of Santa Catarina (UFSC) Department of Computer Science, and his bachelor's degree from the Santa Catarina State University in Data Processing. Security consultant at LockNet Security Solutions, in the software area, between 1999–2003, having worked on projects of national companies (private and public) and multinationals. At the EPUSP he coordinated several research projects with Ericsson Research and worked on projects with RNP and the European Community (FP7). He currently conducts research on cloud computing, computer network security, and ICT sustainability. Currently, he assists in the coordination of the LabP2D (Laboratory of Parallel and Distributed Processing).

Maurício Aronne Pillon Professor of Distributed Systems, and Parallel Programming at Santa Catarina State University (UDESC) in Joinville/SC - Brazil. He received his doctorate from the Institut National Polytechnique de Grenoble at France, the master's degree from Pontifical Catholic University of Rio Grande do Sul (PUC-RS), and his bachelor's degree from the Regional University of the Northwest of the State of Rio Grande do Sul in Informatics. Post-doctorate at the Federal University of Rio Grande do Sul (UFRGS), conducted in the Research Group — Parallel and Distributed Processing Group (GPPD). He currently conducts research on cloud computing, computer networks, and energy efficiency. Currently, he assists on the coordination of the LabP2D (Laboratory of Parallel and Distributed Processing) of UDESC which has a private OpenStack cloud.

Guilherme Piêgas Koslovski Professor of Computer Networks and Parallel Programming at Santa Catarina State University (UDESC) in Joinville/SC - Brazil. He received his doctorate from École Normale Supérieure at Lyon/France, the master's degree from Federal University of Santa Maria (UFSM), and his bachelor's degree from the UFSM in Computer Science. Currently, his researches are related to virtual infrastructures, scheduling, software defined networks, and virtualization of computational and communication resources. He is the coordinator of LabP2D (Laboratory of Parallel and Distributed Processing) at UDESC.